

INSTITUTO SUPERIOR DE CIÊNCIAS E TECNOLOGIA DE MOÇAMBIQUE

Curso de Engenharia Informática

3º Ano - Turma: **A**
2026

Sistemas Distribuídos

RELATÓRIO DO TRABALHO PRÁTICO

AgroGest: Sistema de Gestão Agrícola em Java (Jakarta Servlets + JSP)

Nomes	**Código do Estudante**
Muhamad Shahin Riaz	20240568
Adayl Rafik Amad Seni	20240330
Kenny Matsule	20240908

Maputo, Outubro de 2026

Índice

1. Introdução..... 3

1.1 Objectivo geral..... 3

1.2 Objectivos específicos..... 3

1.3 Estrutura do relatório..... 3

2. Análise de requisitos..... 4

2.1 Perfis de utilizador..... 4

2.2 Requisitos funcionais..... 4

2.3 Requisitos não funcionais..... 5

3. Tecnologias utilizadas..... 5

4. Arquitectura do sistema..... 6

4.1 Padrão MVC..... 6

4.2 Visão geral das camadas..... 6

4.3 Fluxo de um pedido..... 6

5. Estrutura do projecto..... 7

5.1 Árvore de pastas..... 7

5.2 Pacotes Java..... 8

6. Modelo de dados..... 8

6.1 Entidades..... 8

6.2 Relações entre entidades..... 9

6.3 Armazenamento em memória..... 9

7. Controladores e rotas..... 9

8. Regras de negócio..... 11

8.1 Venda atómica..... 12

9. Segurança, concorrência e persistência..... 12

9.1 Autenticação e controlo de acesso..... 12

9.2 Concorrência..... 13

9.3 Persistência em ficheiro..... 13

10. Instalação e execução..... 14

10.1 Requisitos..... 14

10.2 Compilar e publicar..... 14
10.3 Servidor Linux com systemd..... 14
10.4 Contas de teste..... 15
11. Conclusão..... 15
11.1 Limitações..... 15
11.2 Trabalho futuro..... 15
Referências bibliográficas..... 16

1. Introdução

O presente relatório descreve o desenvolvimento do **AgroGest**, uma aplicação web de gestão agrícola desenvolvida no âmbito da disciplina de Sistemas Distribuídos. O sistema responde às necessidades de um agricultor da província de Nampula, que precisa de gerir fornecedores, campanhas agrícolas, cultivos e colheitas, vender produtos em stock a clientes e emitir facturas.

A aplicação foi implementada em **Java**, com **Jakarta Servlets** como controladores e **JSP** como vistas, e é publicada no servidor **Apache Tomcat 10.1**. Por restrição do enunciado, não se usa base de dados: os dados ficam em listas em memória e são gravados num ficheiro através da serialização Java.

1.1 Objectivo geral

Desenvolver uma aplicação web em Java, segundo o padrão MVC, que permita gerir a produção agrícola e a venda de insumos, com controlo de acesso por perfil e persistência em ficheiro.

1.2 Objectivos específicos

- Estruturar a aplicação segundo o padrão Model-View-Controller (MVC);
- Implementar servlets como controladores, JSP como vistas e POJOs como modelo de domínio;
- Gerir sessões HTTP para autenticação e carrinho de compras;
- Controlar o acesso às páginas por perfil através de um filtro;
- Usar listas sincronizadas em memória como camada de dados;
- Garantir que uma venda desconta o stock de forma atómica;
- Persistir o estado do sistema em ficheiro com `ObjectOutputStream` e `ObjectInputStream`;
- Empacotar a aplicação num ficheiro WAR e publicá-la no Tomcat.

1.3 Estrutura do relatório

O capítulo 2 apresenta os requisitos e o capítulo 3 as tecnologias usadas. Os capítulos 4 e 5 descrevem a arquitectura e a estrutura de pastas do projecto. O capítulo 6 apresenta o modelo de dados, o capítulo 7 os controladores e as rotas, e o capítulo 8 as regras de negócio. O capítulo 9 trata da segurança, concorrência e persistência, o capítulo 10 da instalação, e o capítulo 11 apresenta a conclusão.

2. Análise de requisitos

2.1 Perfis de utilizador

Tabela 1 – Perfis de utilizador

Perfil	*Descrição*
Administrador	Conta fixa criada no arranque (admin@agro.mz). Gere todo o sistema: catálogo, fornecedores, campanhas, cultivos, colheitas, clientes, vendas e relatórios.
Cliente	Regista-se sozinho (ou é registado pelo administrador). Consulta o catálogo, usa o carrinho, compra e consulta as suas facturas.

2.2 Requisitos funcionais

Tabela 2 – Requisitos funcionais

Código	*Requisito*	*Perfil*
RF01	Autenticar com email e palavra-passe	Ambos
RF02	Registar conta de cliente (nome, email único, palavra-passe, telefone, endereço)	Cliente
RF03	Criar, listar e remover fornecedores	Admin
RF04	Criar, listar, repor stock e remover produtos	Admin
RF05	Criar, fechar, reabrir e remover campanhas agrícolas	Admin
RF06	Registar e remover cultivos por campanha	Admin
RF07	Registar colheitas por cultivo, com cálculo do valor total	Admin
RF08	Listar e ordenar produtos, fornecedores, clientes e vendas	Admin
RF09	Registar clientes e vendas ao balcão	Admin
RF10	Emitir relatórios de clientes (por nome, por volume, por assiduidade, sem compras)	Admin
RF11	Gravar e eliminar manualmente o ficheiro de estado	Admin
RF12	Consultar o catálogo, filtrar por categoria, pesquisar e ordenar	Cliente
RF13	Gerir o carrinho (adicionar, actualizar, remover, esvaziar)	Cliente
RF14	Confirmar a compra e obter factura com IVA de 16%	Cliente
RF15	Consultar o histórico de compras e as facturas	Cliente

2.3 Requisitos não funcionais

Tabela 3 – Requisitos não funcionais

Código	*Requisito*
RNF01	Interface web em português, responsiva, com CSS próprio
RNF02	Arquitectura MVC com código organizado em pacotes por responsabilidade
RNF03	Operações de escrita concorrentes protegidas por sincronização
RNF04	Palavras-passe guardadas com hash (SHA-256), nunca em texto simples
RNF05	Persistência tolerante a falhas (escrita atómica em ficheiro)
RNF06	Compatível com Java 17 e Apache Tomcat 10.1
RNF07	Sem base de dados relacional (restrição do enunciado)

3. Tecnologias utilizadas

Tabela 4 – Tecnologias utilizadas

Tecnologia	*Versão*	*Função no projecto*
Java	17	Linguagem de programação
Jakarta Servlet API	6.0	Controladores (`@WebServlet`), filtros e listeners
JSP + JSTL + EL	JSP 3.1 / JSTL 3.0	Vistas (páginas HTML geradas no servidor)
Apache Tomcat	10.1	Servidor de aplicações (contentor de servlets)
Apache Maven	3.9	Gestão de dependências e empacotamento WAR
HTML5, CSS3, JavaScript	—	Interface e gráficos do dashboard
Serialização Java	—	Persistência do estado em `agrogest_state.ser`
Git / GitHub	—	Controlo de versões e trabalho em grupo

As dependências da Servlet API e da JSP API estão declaradas no `pom.xml` com `scope provided`, porque o próprio Tomcat as fornece em tempo de execução. A biblioteca JSTL é incluída no WAR.

4. Arquitectura do sistema

4.1 Padrão MVC

A aplicação segue o padrão **Model-View-Controller**, que separa os dados, a apresentação e o controlo do fluxo:

Tabela 5 – Camadas MVC

Camada	*Responsabilidade*	*No AgroGest*
Model	Representa os dados e as regras de negócio	Pacotes model, repo e service
View	Apresenta os dados ao utilizador	Ficheiros JSP em WEB-INF/views
Controller	Recebe o pedido HTTP, chama o modelo e escolhe a vista	Servlets nos pacotes servlet e servlet.admin

As JSP ficam dentro de WEB-INF, que o navegador não consegue abrir directamente. Assim, qualquer página passa obrigatoriamente por um servlet.

4.2 Visão geral das camadas

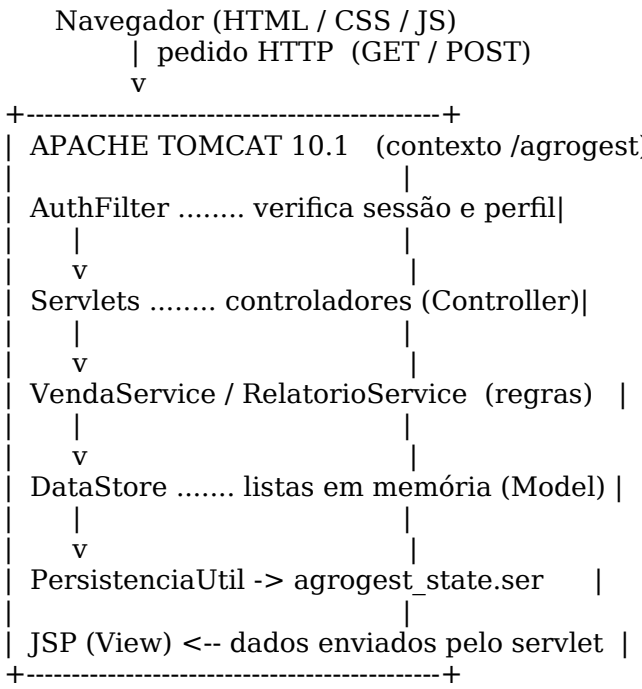


Figura 1 – Camadas da aplicação

4.3 Fluxo de um pedido

Exemplo: um cliente confirma uma compra.

1. O navegador envia POST /compra.
2. O AuthFilter confirma que existe sessão com utilizador autenticado.

3. O CheckoutServlet lê o carrinho da sessão e chama VendaService.efetuarVenda().
4. O serviço verifica o stock de todos os itens, desconta-o, cria a Venda e guarda-a no DataStore.
5. O servlet esvazia o carrinho e redirecciona para /factura?id=N (padrão Post-Redirect-Get, que evita repetir a compra ao actualizar a página).
6. O FacturaServlet obtém a venda e envia-a para factura.jsp, que gera o HTML.
7. Se faltar stock, o serviço lança uma excepção e o servlet volta a mostrar o carrinho com a mensagem de erro.

5. Estrutura do projecto

5.1 Árvore de pastas

```
agrogest/
├── pom.xml                  configuração Maven (dependências, WAR)
├── README.md                descrição e instruções do projecto
├── run.bat / stop.bat       arrancar e parar no Windows
├── docs/
│   ├── ENUNCIADO_TRABALHO_PRATICO.md
│   ├── MANUAL_DE_UTILIZACAO.pdf
│   └── tomcat.service
└── src/main/
    ├── java/mz/ac/istcem/agrogest/
    │   ├── model/           entidades (POJOs Serializable)
    │   ├── repo/            DataStore e EstadoSistema
    │   ├── service/         regras de negócio
    │   ├── servlet/         controladores do cliente e autenticação
    │   │   └── admin/       controladores do administrador
    │   ├── filter/          AuthFilter (controlo de acesso)
    │   ├── listener/        AppListener (arranque e encerramento)
    │   └── util/            hash, persistência, gráficos
    └── webapp/
        ├── index.jsp        redirecciona para o login
        ├── login.jsp        início de sessão
        ├── registo.jsp      registo de cliente
        ├── erro403.jsp      acesso restrito
        ├── css/style.css    estilos da interface
        ├── js/graficos.js   gráficos do dashboard
        ├── img/             logótipos e fotografias dos produtos
        └── WEB-INF/
            ├── web.xml      configuração (UTF-8, sessão de 30 min)
            └── views/
                ├── _partials/   cabeçalho, menu lateral, barra do cliente
                ├── admin/       9 páginas do administrador
                └── cliente/     catálogo, carrinho, factura, compras
```

Figura 2 – Estrutura de pastas do projecto

5.2 Pacotes Java

Tabela 6 – Pacotes e classes Java

Pacote	*Classes*	*Responsabilidade*
model	Utilizador, Administrador, Cliente, Fornecedor, Produto, Campanha, Cultivo, Colheita, Venda, VendaItem	Entidades do domínio
repo	DataStore, EstadoSistema	Armazenamento em memória e fotografia do estado para gravar
service	VendaService, RelatorioService	Regras de negócio: vendas e relatórios
servlet	LoginServlet, LogoutServlet, RegistoServlet, CatalogoServlet, CarrinhoServlet, CheckoutServlet, FacturaServlet, MinhasComprasServlet	Controladores de autenticação e do cliente
servlet.admin	AdminDashboardServlet, ProdutoServlet, FornecedorServlet, CampanhaServlet, CultivoServlet, ColheitaServlet, ClienteAdminServlet, VendaAdminServlet, RelatorioServlet, PersistenciaServlet	Controladores do administrador
filter	AuthFilter	Controlo de acesso por sessão e perfil
listener	AppListener	Carrega o estado no arranque, cria dados iniciais e grava no encerramento
util	PasswordUtil, PersistenciaUtil, GraficoUtil	Hash SHA-256, gravação em ficheiro e dados para gráficos

No total, o projecto tem 37 classes Java (cerca de 2 000 linhas) e 17 ficheiros JSP.

6. Modelo de dados

6.1 Entidades

Todas as entidades implementam `Serializable`, para poderem ser gravadas em ficheiro. `Administrador` e `Cliente` herdam de `Utilizador`.

Tabela 7 – Entidades e atributos

Classe	*Atributos*
Utilizador	id, nome, email, passwordHash, perfil (ADMIN / CLIENTE), ativo
Administrador	herda de Utilizador

Classe	*Atributos*
Cliente	herda de Utilizador + telefone, endereco
Fornecedor	id, nome, tipo (SEMENTES / MAQUINARIA / FERTILIZANTES), contacto, provincia
Produto	id, nome, categoria (SEMENTE / FERTILIZANTE / MAQUINARIA / FERRAMENTA), fornecedorId, fornecedorNome, preco, quantidade, unidade, descricao, fotoUrl, validade
Campanha	id, nome, inicio, fim, aberta
Cultivo	id, campanhaId, cultura, variedade, hectares, dataPlantio
Colheita	id, cultivoId, data, quantidadeKg, precoPorKg, valorTotal
Venda	id, numeroFactura, clienteId, clienteNome, clienteEndereco, clienteTelefone, data, itens, subtotal, iva, total
VendaItem	produtoId, produtoNome, quantidade, precoUnitario, subtotal

6.2 Relações entre entidades

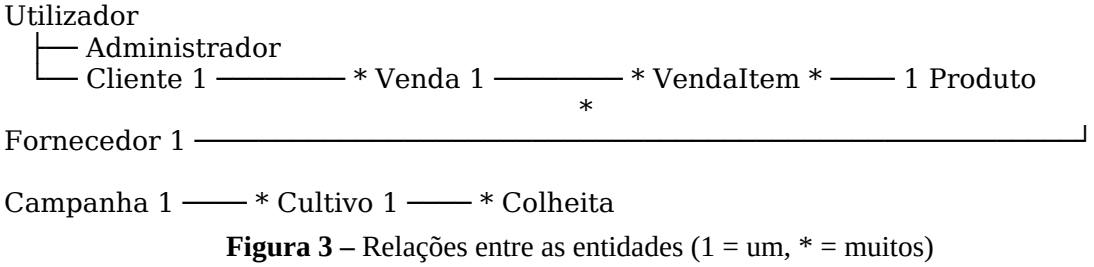


Figura 3 – Relações entre as entidades (1 = um, * = muitos)

As relações são feitas por identificador (por exemplo, `Cultivo.campanhaId`), como numa base de dados. A `Venda` guarda também uma cópia do nome, endereço e telefone do cliente e do preço de cada produto no momento da compra. Assim, a factura não muda se o cliente ou o preço forem alterados depois.

6.3 Armazenamento em memória

A classe `DataStore` segue o padrão **Singleton**: existe uma única instância, partilhada por todos os servlets e obtida com `DataStore.getInstance()`. Contém uma lista por entidade, criada com `Collections.synchronizedList(new ArrayList<>())`, e um contador `AtomicInteger` por entidade para gerar identificadores sequenciais. Também tem o contador do número de factura.

7. Controladores e rotas

Cada servlet é registado com a anotação `@WebServlet`, que indica o endereço que atende. Os formulários enviam um campo escondido `acao`, que diz ao servlet que operação executar.

Tabela 8 – Servlets, rotas e acções

Servlet	*Rota*	*Métodos*	*Acções / função*
LoginServlet	/login	POST	Valida as credenciais e cria a sessão
LogoutServlet	/logout	GET	Termina a sessão
RegistoServlet	/registo	POST	Regista um novo cliente
CatalogoServlet	/catalogo	GET	Lista os produtos com filtro, pesquisa e ordenação
CarrinhoServlet	/carrinho/*	GET, POST	adicionar, atualizar, remover, limpar
CheckoutServlet	/compra	POST	Confirma a compra e cria a factura
FacturaServlet	/factura	GET	Mostra uma factura (só ao dono ou ao admin)
MinhasComprasServlet	/minhas-compras	GET	Histórico de compras do cliente
AdminDashboardServlet	/admin/dashboard	GET	Indicadores e gráficos
ProdutoServlet	/admin/produtos	GET, POST	criar, repor, remover
FornecedorServlet	/admin/fornecedores	GET, POST	criar, remover
CampanhaServlet	/admin/campanhas	GET, POST	criar, fechar, reabrir, remover
CultivoServlet	/admin/cultivos	GET, POST	criar, remover
ColheitaServlet	/admin/colheitas	GET, POST	criar
ClienteAdminServlet	/admin/clientes	GET, POST	criar, remover
VendaAdminServlet	/admin/vendas	GET, POST	Histórico de vendas e venda ao balcão
RelatorioServlet	/admin/relatorios	GET	Relatórios de clientes
PersistenciaServlet	/admin/persistencia	POST	gravar, eliminar

Os servlets seguem o padrão **Post-Redirect-Get**: depois de um POST que altera dados, respondem com um redireccionamento para uma página GET. Assim, actualizar a página no navegador não repete a operação.

8. Regras de negócio

Tabela 9 – Regras de negócio e respectiva implementação

Regra	*Descrição*	*Onde é implementada*
RN01	Não vender mais do que o stock disponível	CarrinhoServlet (limita a quantidade) e VendaService (verifica antes de vender)
RN02	A venda desconta o stock de todos os itens de forma atómica (tudo ou nada)	VendaService.efetuarVenda(), dentro de synchronized (store)
RN03	O cliente só vê as suas facturas; o admin vê todas	FacturaServlet e MinhasComprasServlet
RN04	Rotas /admin/* exclusivas do administrador	AuthFilter (redirecciona para erro403.jsp)
RN05	Catálogo, carrinho, compra, factura e histórico exigem sessão	AuthFilter (redirecciona para o login)
RN06	Identificadores sequenciais e únicos	Contadores AtomicInteger no DataStore
—	Não remover campanha com cultivos	CampanhaServlet
—	Não remover cultivo com colheitas	CultivoServlet
—	Não remover cliente com vendas	ClienteAdminServlet
—	Email de cliente único	RegistoServlet e ClienteAdminServlet
—	IVA de 16% e valor total da colheita calculados automaticamente	Venda.calcularTotais() e Colheita
—	Campos obrigatórios (nome)	RegistoServlet, ProdutoServlet, FornecedorServlet, CampanhaServlet, CultivoServlet e ClienteAdminServlet
—	Preços e quantidades não negativos	ProdutoServlet
—	Categorias de produto: SEMENTE, FERTILIZANTE, MAQUINARIA, FERRAMENTA	ProdutoServlet
—	Tipo de fornecedor: SEMENTES, MAQUINARIA, FERTILIZANTES	FornecedorServlet
—	Email válido e palavra-passe com pelo menos 4 caracteres	RegistoServlet e ClienteAdminServlet
—	Data de fim da campanha não anterior à data de início	CampanhaServlet

Regra	*Descrição*	*Onde é implementada*
—	Data da colheita não anterior à data de plantio	ColheitaServlet
—	Dados inválidos mostram uma mensagem de erro em vez de um erro HTTP 500	Servlets do administrador, CarrinhoServlet
—	Identificador de factura inválido devolve HTTP 400	FacturaServlet

8.1 Venda atómica

O método efetuarVenda funciona em duas passagens. Na primeira, verifica se há stock para **todos** os itens; se faltar algum, lança IllegalStateException("Stock insuficiente para: …") antes de alterar qualquer valor. Só na segunda passagem desconta as quantidades. Depois cria a venda com número de factura sequencial, calcula o subtotal, o IVA (16%) e o total, e guarda-a. Antes de tudo, rejeita carrinhos vazios (IllegalStateException("Carrinho vazio")). Todo o processo corre dentro de um bloco synchronized (store), isto é, com um lock sobre o singleton DataStore, partilhado por todas as instâncias de VendaService e pelo ProdutoServlet. Assim, duas compras simultâneas não podem vender a mesma unidade de stock, nem uma reposição ou remoção de produto pode intercalar-se a meio da venda. Depois de concluída, a venda é gravada em ficheiro.

```
public Venda efetuarVenda(Cliente cliente, List<VendaItem> itens) {
    synchronized (store) {                          // lock do singleton DataStore
        Venda v = efetuarVendaInterno(cliente, itens);
        PersistenciaUtil.gravar();
        return v;
    }
}

private Venda efetuarVendaInterno(Cliente cliente, List<VendaItem> itens) {
    if (itens == null || itens.isEmpty())
        throw new IllegalStateException("Carrinho vazio");
    for (VendaItem item : itens) {                  // 1.ª passagem: verificar
        Produto p = store.findProduto(item.getProdutoId());
        if (p == null || p.getQuantidade() < item.getQuantidade())
            throw new IllegalStateException("Stock insuficiente para: " + item.getProdutoNome());
    }
    for (VendaItem item : itens) {                  // 2.ª passagem: descontar
        Produto p = store.findProduto(item.getProdutoId());
        p.setQuantidade(p.getQuantidade() - item.getQuantidade());
    }
    Venda v = new Venda(); ... v.calcularTotais(); store.vendas.add(v);
    return v;
}
```

Figura 4 – Excerto de VendaService.efetuarVenda()

9. Segurança, concorrência e persistência

9.1 Autenticação e controlo de acesso

No login, o LoginServlet calcula o hash SHA-256 da palavra-passe introduzida (PasswordUtil.hash) e compara-o com o hash guardado. Se coincidirem, grava na sessão o utilizador, o perfil e o nome. A sessão expira após 30 minutos de inactividade (web.xml).

O AuthFilter (@WebFilter) intercepta as rotas protegidas antes de chegarem ao servlet:

- sem sessão ou sem utilizador → redirecciona para /login.jsp;
- rota /admin/* com perfil diferente de ADMIN → redirecciona para /erro403.jsp;
- caso contrário → deixa o pedido continuar (chain.doFilter).

9.2 Concorrência

O Tomcat atende vários pedidos ao mesmo tempo, em threads diferentes, e todos partilham o mesmo DataStore. Para evitar inconsistências:

- as listas são Collections.synchronizedList, o que torna seguras as operações individuais (adicionar, remover);
- os métodos de pesquisa do DataStore são synchronized;
- a venda (VendaService.efetuarVenda), o registo de clientes (RegistoServlet) e a criação, reposição de stock e remoção de produtos (ProdutoServlet) usam o mesmo lock, synchronized (store), sobre o singleton DataStore, pelo que nunca se executam em simultâneo;
- os identificadores usam AtomicInteger.getAndIncrement(), que nunca dá o mesmo número a dois pedidos.

9.3 Persistência em ficheiro

A classe PersistenciaUtil grava o estado no ficheiro agrogest_state.ser, na pasta do Tomcat (catalina.base). A gravação é automática após cada alteração (venda, produto, campanha, cliente, etc.), e também ocorre quando o Tomcat é desligado e quando o administrador clica em "Gravar":

- **Gravar:** o DataStore cria um EstadoSistema (cópia de todas as listas e contadores), que é escrito com ObjectOutputStream para um ficheiro .tmp. Antes de o substituir, o ficheiro anterior é copiado para agrogest_state.ser.bak. Só depois o .tmp é renomeado para o nome final. Assim, uma falha a meio da gravação não estraga o ficheiro anterior (escrita atómica) e fica sempre uma cópia de segurança.
- **Carregar:** lê o ficheiro com ObjectInputStream e repõe as listas e os contadores. Se o ficheiro .ser estiver corrompido, é copiado para agrogest_state.ser.corrompido e é carregada a cópia agrogest_state.ser.bak.
- **Eliminar:** apaga o ficheiro de estado e a respectiva cópia .bak.

```
Tomcat arranca ──> AppListener.contextInitialized()
                   ├─ PersistenciaUtil.carregar() (se o ficheiro existir)
                   │    └─ se corrompido: copia para .corrompido e carrega .bak
                   └─ cria admin e dados de demonstração (se não existirem)

Venda, produto, campanha, cliente... ──> PersistenciaUtil.gravar()  (automático)

Admin clica "Gravar" ──> PersistenciaServlet ──> PersistenciaUtil.gravar()

Tomcat é desligado ──> AppListener.contextDestroyed()
                       └─ PersistenciaUtil.gravar()
                          1. escreve agrogest_state.ser.tmp
                          2. copia o .ser anterior para agrogest_state.ser.bak
                          3. renomeia o .tmp para agrogest_state.ser
```

Figura 5 – Ciclo de vida da persistência

10. Instalação e execução

10.1 Requisitos

Java JDK 17 ou superior, Apache Maven 3.9 e Apache Tomcat 10.1.

10.2 Compilar e publicar

```
cd agrogest
mvn clean package                  # gera target/agrogest.war
cp target/agrogest.war ../tomcat10/webapps/
../tomcat10/bin/startup.sh         # Windows: run.bat
# abrir http://localhost:8080/agrogest
../tomcat10/bin/shutdown.sh        # parar (grava o estado) | Windows: stop.bat
```

Figura 6 – Comandos de compilação e execução

10.3 Servidor Linux com systemd

```
sudo cp target/agrogest.war /opt/tomcat/webapps/
sudo cp docs/tomcat.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable tomcat
sudo systemctl restart tomcat
sudo systemctl status tomcat
```

Figura 7 – Deploy com systemd

O ficheiro docs/tomcat.service define o serviço systemd do Tomcat. Por omissão, o Tomcat atende no porto 8080. Para usar outro porto, edite o ficheiro conf/server.xml e altere o atributo port do conector HTTP, por exemplo <Connector port="8081" protocol="HTTP/1.1" ... />; depois reinicie o serviço (sudo systemctl restart tomcat) e abra http://localhost:8081/agrogest.

10.4 Contas de teste

Tabela 10 – Contas criadas no arranque

Perfil	*Email*	*Palavra-passe*
Administrador	admin@agro.mz	admin234
Cliente	cliente@agro.mz	cliente123
Cliente	joao@teste.mz	1234

11. Conclusão

O AgroGest cumpre os requisitos do enunciado: o administrador gere fornecedores, produtos, campanhas, cultivos, colheitas, clientes e vendas, e consulta relatórios; o cliente regista-se, compra através do carrinho e obtém facturas com IVA. A separação em camadas MVC, com servlets, JSP, serviços e um repositório em memória, tornou o código organizado e fácil de alterar.

O trabalho permitiu aplicar conceitos centrais de Sistemas Distribuídos num ambiente cliente-servidor: pedidos HTTP concorrentes, sessões, filtros, sincronização de recursos partilhados e persistência do estado.

11.1 Limitações

- Os dados ficam em memória, o que limita a quantidade de dados e impede usar vários servidores ao mesmo tempo;
- O SHA-256 sem "salt" é uma protecção básica para palavras-passe.

11.2 Trabalho futuro

- Substituir as listas em memória por uma base de dados relacional (por exemplo, PostgreSQL);
- Usar um algoritmo próprio para palavras-passe (BCrypt ou PBKDF2);
- Exportar facturas e relatórios em PDF.

Referências bibliográficas

- Eclipse Foundation. *Jakarta Servlet Specification, Version 6.0*. Disponível em: https://jakarta.ee/specifications/servlet/6.0/
- Eclipse Foundation. *Jakarta Server Pages Specification, Version 3.1*. Disponível em: https://jakarta.ee/specifications/pages/3.1/
- The Apache Software Foundation. *Apache Tomcat 10.1 Documentation*. Disponível em: https://tomcat.apache.org/tomcat-10.1-doc/
- The Apache Software Foundation. *Apache Maven Documentation*. Disponível em: https://maven.apache.org/guides/
- Oracle. *Java SE 17 API Documentation* (java.io.Serializable, java.util.concurrent.atomic.AtomicInteger). Disponível em: https://docs.oracle.com/en/java/javase/17/docs/api/
- Coulouris, G., Dollimore, J., Kindberg, T. & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5.ª ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Enunciado do Trabalho Prático — AgroGest, Sistemas Distribuídos, ISCTEM, 2026.